Software Requirements Specification

Project: Campus Recruit Analyzer (CRA)

Prepared by:
Kartik Kaushal & Manjushwar Khairkar
Dr. B.R. Ambedkar National Institute of Technology, Jalandhar

Date: 20.03.26 | Version: 1.0

1. Introduction

1.1 Purpose

- The placement season for third and fourth year students is highly demanding.
- Numerous companies visit campus to recruit top talent from the college.
- Students apply to many companies but lack structured, company-specific preparation guidance within short timeframes.
- Students often prepare for topics that are irrelevant to specific company interviews or Online Assessments (OAs).
- This results in suboptimal interview performance.
- CRA addresses this gap by providing targeted, company-specific preparation roadmaps.

1.2 Document Conventions

- This document follows the IEEE 830-1998 standard for Software Requirements Specifications.
- Requirements are written as "The system shall..." to distinguish mandatory requirements from descriptive text.
- Informational content uses standard declarative sentences.
- Requirements are grouped by feature in Section 4 and by non-functional category in Section 5.
- Each requirement is assigned a local identifier (REQ-1, REQ-2, etc.) scoped within its parent feature section.
- Priority levels (High, Medium-High, Medium) are stated at the feature level and inherited by all requirements under it unless explicitly overridden.
- Section headings use a hierarchical numbering scheme.
- Bold text is used for section titles and key terms introduced for the first time.
- Calibri font is used consistently throughout this document.

1.3 Intended Audience and Reading Suggestions

This SRS is intended for the following audiences:

- Developers of the software
- Supervising faculty

- Product managers
- Users (NITJ students)

1.4 Product Scope

- The software assists students preparing for multiple interviews at different companies.
- It helps students secure internships or jobs by allowing them to focus entirely on preparation.
- Students are relieved from the burden of researching question patterns themselves.
- The college can provide this as a resource to help students achieve the best packages from campus placements.

1.5 References

- IEEE Std 830-1998 – IEEE Recommended Practice for Software Requirements Specifications. Available at: https://ieeexplore.ieee.org/document/720574
- PyQt6 Documentation – Riverbank Computing, Version 6.x. Available at: https://www.riverbankcomputing.com/static/Docs/PyQt6/
- MySQL 8.0 Reference Manual – Oracle Corporation, 2023. Available at: https://dev.mysql.com/doc/refman/8.0/en/
- GeeksforGeeks – Company Interview Experiences. Available at: https://www.geeksforgeeks.org/company-interview-corner/
- OWASP Top Ten – Web Application Security Risks. OWASP Foundation, 2021. Available at: https://owasp.org/www-project-top-ten/
- NIT Jalandhar Training and Placement Cell – Official Placement Records. Available internally via the institute's placement office.

2. Overall Description

2.1 Product Perspective

- Campus Recruit Analyzer (CRA) is a standalone, college-specific desktop application built in Python using PyQt6.
- It is developed exclusively for students of Dr. B.R. Ambedkar National Institute of Technology, Jalandhar (NIT Jalandhar).
- It is a new, self-contained system that does not form part of any larger existing system.
- It is intentionally non-scalable and scoped exclusively for use within a single institution.
- Platforms like LeetCode, InterviewBit, and GeeksforGeeks offer general preparation material but none provide college-contextualized, company-specific guidance.
- CRA fills this gap by combining historical NITJ placement data, public interview experiences, and AI-driven summarization.
- On first launch, the user provides institutional email credentials for local authentication and email parsing.
- No web server or remote hosting is involved.
- The system interacts with an LLM API for summarization and a local MySQL database for persistent storage.

2.2 Product Functions

Student-Facing Functions

- Email credential input on first launch for authentication and email parsing
- Viewing and filtering companies visiting NITJ for placements or internships
- Viewing company-specific preparation roadmaps including relevant topics, question types, interview rounds, and difficulty levels
- AI-generated summaries of job roles, required skills, and recommended preparation strategies
- Progress tracking to monitor topics covered and readiness level
- Personalized recommendations based on registered companies and academic profile

Admin-Facing Functions

- Managing the company database including adding, updating, and removing company records
- Managing the question and topic database
- Viewing analytics on student engagement and preparation trends
- Controlling access and maintaining data quality

System-Level Functions

- Parsing placement-related emails using provided credentials to extract company and OA information
- Aggregating and processing historical NITJ placement data
- Generating structured preparation summaries using LLM integration
- Storing and retrieving all data through a local relational database

2.3 User Classes and Characteristics

Pre-Placement Students (Primary Users)

- Third and fourth year undergraduate or postgraduate students at NITJ actively participating in placements or internships.
- They are technically literate but may not be experienced with placement processes.
- They will interact with the system frequently to browse companies, view preparation material, and track progress.
- This group has the highest usage frequency and broadest range of technical backgrounds.

Veteran/Experienced Students (Secondary Users)

- Students with prior internship or placement experience.
- Less likely to rely heavily on the system; may use it selectively (e.g., to check OA patterns or interview structures for a specific company).

System Administrators

- Likely faculty coordinators or student developers responsible for maintaining the system.
- Responsible for keeping the company database accurate and up to date, managing question banks, and monitoring system health.
- Expected to be technically proficient.

Placement Cell / Training and Placement Office (TPO)

- Institutional stakeholders who may use the system to understand student preparation trends.

- Help identify gaps and plan placement-related workshops or resources.
- Interact primarily with the analytics dashboard and do not modify academic content directly.

2.4 Operating Environment

Application Environment

- Operating System: Windows 10/11 or Linux (Ubuntu 20.04+)
- Runtime: Python 3.10+ with PyQt6
- Database: MySQL 8.0 (local installation)
- Deployment: Installed locally on the student's machine; no server or cloud hosting required
- Network: Internet connection required only for LLM API calls and email parsing

External Dependencies

- LLM API (e.g., OpenAI GPT or equivalent) for AI-based summarization and recommendations
- IMAP/SMTP access to the user's institutional email account for email parsing
- MySQL 8.0 installed locally on the host machine

2.5 Design and Implementation Constraints

- Non-Scalable by Design: CRA is intentionally scoped as a single-user desktop application. All processing and data storage occur locally.
- Email Credential Requirement: The application requires valid institutional email credentials on first launch. These are stored locally in encrypted form and used exclusively for parsing placement-related emails.
- Limited Historical Data: Structured, digitized historical placement data for NITJ is limited. Much of the data may require manual curation, affecting accuracy in early versions.
- AI API Cost: Usage of external LLM APIs incurs per-request costs. The system minimizes unnecessary API calls through caching and batched processing.
- Single Institution Scope: Version 1.0 is scoped exclusively for NIT Jalandhar. The architecture should be modular enough to allow future extension, but no multi-tenancy is required now.
- No Web or Mobile Interface: CRA is a desktop application only in Version 1.0.
- Regulatory Constraints: All student data must be handled per applicable data privacy norms. No PII shall be shared with external APIs without anonymization.
- Technology Stack Constraint: The project is constrained to use Python with PyQt6 for the GUI and MySQL for the database.
- Internet Dependency: An active internet connection is required for LLM API access and email parsing. Offline functionality for these features is not supported in Version 1.0.

2.6 User Documentation

- User Manual: Step-by-step guide for students covering first-launch credential setup, company browsing, preparation material, and progress tracking.
- Admin Manual: Guide for administrators covering company and question database management and use of the analytics dashboard.
- Installation Guide: Instructions for setting up the application locally, including Python environment setup, dependency installation, and MySQL configuration.
- README File: A concise quick-start guide hosted in the project repository covering basic setup and usage.

- In-App Help Tooltips: Contextual guidance embedded within the interface to assist first-time users.

2.7 Assumptions and Dependencies

Assumptions

- Students have access to a Windows or Linux machine with Python 3.10+ installed.
- Students have a valid NITJ institutional email account accessible via IMAP.
- The placement cell will provide or validate historical company visit data for NITJ to seed the initial database.
- Interview experiences and OA question patterns from public platforms are assumed to be sufficiently representative of actual interview processes.
- Users are assumed to be honest in their profile setup, particularly regarding year of study and registered companies.
- The LLM API used for summarization is assumed to be available and accessible during peak placement season.

Dependencies

- The system depends on access to an active LLM API subscription for core summarization and recommendation features. Loss of API access will degrade these features.
- The system depends on MySQL 8.0 or compatible database software being correctly installed and configured locally.
- GUI functionality depends on PyQt6 being correctly installed in the Python environment.

3. External Interface Requirements

CRA provides a desktop GUI built with PyQt6, accessible only as a locally installed application.

3.1 User Interfaces

Student Interface

- First-Launch Credential Setup Screen (email and password input)
- Main Dashboard
- Company List and Filter View
- Company Preparation Details Screen
- Topic Recommendation View per Company
- Progress Tracking View
- Profile Management Screen

Administrator Interface

- Admin Login Screen
- Company Management Panel
- Question/Topic Database Management
- Analytics Dashboard

3.2 Hardware Interfaces

3.2.1 Supported Hardware Platforms

- Desktop computers and laptops running Windows 10/11 or Linux (Ubuntu 20.04+).

3.2.2 Client Hardware Requirements

- Device capable of running Python 3.10+ and PyQt6
- Minimum 4 GB RAM recommended
- Minimum 500 MB free disk space for application and local database
- Internet connection required for LLM API and email parsing

3.2.3 Server Hardware Requirements

- Not applicable. CRA operates entirely on the local machine with no server-side deployment.

3.3 Software Interfaces

3.3.1 Operating System

- Windows 10 / 11 (64-bit)
- Linux – Ubuntu 20.04 LTS or equivalent

3.3.2 Desktop Application Interface

- The application is implemented using PyQt6.
- All GUI components, windows, dialogs, and widgets are rendered natively via the Qt framework.
- No web browser is required.

3.3.3 Database Interface

- The system shall interface with MySQL 8.0 installed locally.
- Data exchanged includes student profiles, company records, interview rounds, question banks, preparation topics, and progress tracking data.
- Communication shall occur through PyMySQL or SQLAlchemy connectors using SQL queries.

3.3.4 Backend / Application Framework

- The system shall use Python 3.10+ for all application logic.
- Services include authentication handling, session management, data retrieval and storage, email parsing, and recommendation logic execution.
- PyQt6 handles all GUI rendering and event management.

3.3.5 External Libraries and Tools

- PyQt6 – Desktop GUI framework
- SQLAlchemy / PyMySQL – Database ORM and connector
- imaplib / email – Python standard library modules for email parsing
- requests / httpx – HTTP client for LLM API calls
- JSON – Data exchange format for LLM API communication

- All inter-component communication shall use structured data passed through Python function calls or local database transactions.
- No global shared memory mechanisms shall be used.

3.4 Communications Interfaces

3.4.1 Network Protocols

- The system shall use HTTPS (TLS 1.2 or higher) for LLM API communication.
- The system shall use IMAP (port 993, SSL) for reading placement emails from the user's inbox.
- All sensitive data transmission shall occur over encrypted channels.

3.4.2 Email Parsing

- On first launch, the user provides institutional email credentials.
- The system uses IMAP to connect to the mail server and reads placement-related emails.
- Extracts structured information such as company name, OA details, and deadlines.
- Credentials are stored locally in AES-256 encrypted form and are never transmitted to any endpoint other than the institutional email provider.

3.4.3 LLM API Communication

- The system makes HTTPS requests to the configured LLM API endpoint to generate preparation summaries.
- All personally identifiable information is stripped before any data is sent to the API.

3.4.4 Security Requirements

- All user credentials shall be encrypted before storage.
- Email credentials shall be stored locally using AES-256 encryption.
- Passwords shall be hashed using industry-standard hashing algorithms.
- All LLM API communication shall occur over HTTPS.
- Session tokens shall expire after a defined timeout period.
- No credentials shall ever be transmitted to any endpoint other than the user's institutional email provider.

3.4.5 Performance Requirements

- Application launch time shall not exceed 5 seconds on recommended hardware.
- Local database queries shall return results within 1 second under normal conditions.
- LLM API response time is network-dependent; the UI shall display a loading indicator during API calls.
- UI interactions shall respond within 200 milliseconds for all local operations.

4. System Features

4.1 Company-Specific Preparation Roadmap Generation

4.1.1 Description and Priority

- Priority: High
- When a student selects a company they have applied for, the system generates a structured preparation roadmap.
- The roadmap covers interview rounds, frequently tested topics, difficulty levels, and an AI-generated preparation summary.
- This is the core feature of CRA.

4.1.2 Stimulus/Response Sequences

- Student selects a company → System displays round-wise breakdown, topic list, and AI-generated preparation summary.
- Cached data is outdated → System triggers a fresh LLM API call and updates the summary.
- No historical NITJ data exists for a company → System falls back to a general role-based preparation guide.

4.1.3 Functional Requirements

- REQ-1: The system shall maintain a local database of companies that have visited NITJ, including job role, package, year of visit, and round structure.
- REQ-2: The system shall display a round-wise breakdown of the interview process for each company.
- REQ-3: The system shall use an LLM API to generate a concise, actionable preparation summary based on aggregated data.
- REQ-4: LLM-generated summaries shall be cached locally and refreshed at a configurable interval or upon admin request.
- REQ-5: When no NITJ-specific data is available, the system shall fall back to a general role-based preparation guide.

4.2 Student Progress Tracking

4.2.1 Description and Priority

- Priority: Medium-High
- Students can mark topics as completed, set preparation goals, and view a visual summary of their readiness across all registered companies.

4.2.2 Stimulus/Response Sequences

- Student marks a topic as complete → System updates the local database and recalculates completion percentage in real time.
- Student opens dashboard → System displays progress bars and completion status for all registered companies.
- Student deregisters from a company → System archives associated progress data and removes it from the active list.

4.2.3 Functional Requirements

- REQ-1: The system shall allow students to mark individual topics and tasks as complete or incomplete.
- REQ-2: The system shall calculate and display a preparation completion percentage per company.
- REQ-3: The dashboard shall show a consolidated progress view across all registered companies.

- REQ-4: Progress data shall persist in the local database across application sessions.

4.3 Company and Question Database Management

4.3.1 Description and Priority

- Priority: High
- Administrators can add, edit, and delete company records, interview round details, and question/topic entries through a dedicated admin panel.

4.3.2 Stimulus/Response Sequences

- Admin navigates to Company Management Panel → System displays all company records with options to add, edit, or delete.
- Admin submits a new company record → System validates input, stores the record, and makes it immediately visible to students.
- Admin updates the question bank → System updates records and invalidates related cached LLM summaries.

4.3.3 Functional Requirements

- REQ-1: The system shall provide full CRUD functionality for company records via the admin interface.
- REQ-2: Each company record shall include company name, job roles, years of NITJ visits, package offered, round structure, and topic tags.
- REQ-3: The system shall provide an interface to add, edit, categorize, and delete question and topic bank entries.
- REQ-4: The system shall invalidate cached LLM summaries when underlying data is modified.
- REQ-5: The admin panel shall be accessible only to authenticated users with administrator privileges.

4.4 AI-Powered Role Summarization

4.4.1 Description and Priority

- Priority: High
- The system provides AI-generated summaries of job roles covering responsibilities, required skills, and preparation differences across roles (e.g., SDE vs. Data Analyst).

4.4.2 Stimulus/Response Sequences

- Student selects a job role on a company screen → System displays an AI-generated role summary with required skills and interview focus areas.
- Student browses the general role library → System lists all common NITJ placement roles with individual summaries.

4.4.3 Functional Requirements

- REQ-1: The system shall maintain a local library of job roles commonly offered during NITJ placements.

- REQ-2: For each role, the system shall provide an AI-generated summary covering responsibilities, required skills, and interview focus areas.
- REQ-3: Role summaries shall be cached locally and linked from the company preparation screen for contextual access.

4.5 Email Parsing and Company Detection

4.5.1 Description and Priority

- Priority: High
- On first launch, the user provides institutional email credentials.
- The system parses placement-related emails to automatically extract company names, OA details, and deadlines.
- Extracted data pre-populates the preparation dashboard.

4.5.2 Stimulus/Response Sequences

- User provides email credentials on first launch → System authenticates via IMAP and scans the inbox for placement emails.
- Placement email detected → System extracts structured data (company name, role, deadline) and stores it locally.
- No placement emails found → System notifies the user and proceeds with manual company selection.

4.5.3 Functional Requirements

- REQ-1: The system shall prompt for institutional email credentials on first launch.
- REQ-2: The system shall connect to the email server via IMAP and scan for placement-related emails.
- REQ-3: The system shall parse detected emails to extract company name, role, OA date, and deadline information.
- REQ-4: Extracted data shall be stored in the local database and presented to the student on the dashboard.
- REQ-5: Credentials shall be stored locally in AES-256 encrypted form and never transmitted to any external system other than the institutional email server.

4.6 Analytics Dashboard

4.6.1 Description and Priority

- Priority: Medium
- An admin-facing dashboard providing insights into student engagement, most-viewed companies, frequently accessed topics, and overall preparation trends.
- Helps the placement cell identify gaps and plan resources.

4.6.2 Stimulus/Response Sequences

- Admin opens the Analytics Dashboard → System aggregates locally stored usage data and displays visualizations of company views, topic access frequency, and student progress distribution.

4.6.3 Functional Requirements

- REQ-1: The system shall log anonymized student interaction events including screen views, topic completions, and session durations in the local database.
- REQ-2: The dashboard shall display aggregated metrics including top-viewed companies, most-accessed topics, and student progress distribution.
- REQ-3: Analytics shall be accessible only to users with administrator or placement cell privileges.
- REQ-4: No personally identifiable information shall be displayed; all metrics shall be aggregated or anonymized.

5. Other Non-Functional Requirements

5.1 Data Encryption and Integrity

- Student personal information and email credentials must remain safe and confidential.
- All sensitive data stored locally, including email passwords, shall be encrypted using AES-256.
- The local MySQL database shall use access controls to prevent unauthorized reads or writes.

5.2 Safety Requirements

- The system shall prevent accidental data deletion through confirmation dialogs.
- Administrative deletion of company records shall require explicit confirmation.
- The system shall maintain regular automated local database backups.
- Recovery procedures shall be documented and tested quarterly.

5.3 Security Requirements

5.3.1 Authentication

- All users shall authenticate using a unique username/email and password.
- Passwords shall meet minimum complexity requirements: minimum 8 characters, at least one uppercase letter, at least one numeric digit.
- The system shall lock an account after 5 consecutive failed login attempts.
- Email credentials entered at first launch shall be verified against the IMAP server before being stored locally.

5.3.2 Authorization

The system shall implement role-based access control (RBAC):
Student users may:

- View company preparation material
- Track personal progress
- Update personal profile

Admin users may:

- Add, update, or delete company data
- Modify interview round information
- Access analytics dashboard

5.3.3 Privacy Protection

- Student data shall not be shared with third parties.
- Email credentials shall be stored locally in encrypted form and never sent to any server other than the institutional email provider.
- Only authorized administrators may access aggregated student analytics.
- The system shall comply with institutional data privacy policies.

5.4 Software Quality Attributes

5.4.1 Availability

- As a desktop application with local data storage, the system shall be available whenever the host machine is operational.
- Dependency on external services (LLM API, email server) is limited to specific features; core functionality shall remain accessible offline.

5.4.2 Performance

- Application launch time shall not exceed 5 seconds on recommended hardware.
- Local database queries shall return results within 1 second under standard conditions.
- UI interactions shall respond within 200 milliseconds for all local operations.

5.4.3 Reliability

- System failure rate shall not exceed 1% of total user sessions.
- Data loss tolerance shall be zero for committed local database transactions.

5.4.4 Usability

- A new user shall be able to navigate to company preparation content within 3 clicks after login.
- The interface shall be intuitive and require no formal training.
- Error messages shall clearly describe corrective actions.

5.4.5 Maintainability

- Code shall follow a modular architecture separating GUI, business logic, and data layers.
- All modules and functions shall be documented with docstrings.
- All APIs and interfaces shall include descriptive comments.
- The system shall allow addition of new companies without modifying core application logic.

5.4.6 Portability

- The application shall be deployable on Windows 10/11 and Ubuntu 20.04+ without modification.
- A requirements.txt or setup script shall be provided for reproducible environment setup.

5.4.7 Scalability

- CRA is intentionally non-scalable, designed for single-user, local-machine operation.
- The local MySQL database shall support up to 1,000 student and company records without significant performance degradation.
- No horizontal or cloud scaling is supported or required.

5.5 Business Rules

- Only registered NITJ students may create accounts.
- Students may access only their own progress data.
- Only administrators may modify company preparation data.
- Company data must be approved by faculty before being published.
- Historical company data shall remain archived even after the placement season ends.
- Student accounts shall be deactivated after graduation unless extended by admin approval.
- Analytics reports shall present anonymized data when viewed by faculty.
- Email credentials provided at first launch may be updated by the student at any time from the Profile Management screen.